ALGO PBX · INFRASTRUCTURE GUIDE 1 OF 2

Deploying Algo PBX on a Linux VM

From an empty VirtualBox VM to a running Docker Compose stack and a working first admin login — every command exactly as shipped in this repository's `DEPLOYMENT.md`.

Scope: VM creation → Ubuntu Server install → Docker prep → clone & secrets → build & start → first login.

Companion document: "Configuring Credentials, Dinstar & Going Live" covers `/admin/settings`, the Dinstar UC2000 gateway, and the GoDaddy→Cloudflare domain cutover.

Note on screenshots: VirtualBox/Ubuntu installer screens are captured live as you run them together with this checklist — they can't be generated ahead of time from outside the installer. Diagram boxes below show exactly what each screen looks like and what to click.

Contents

STEP 1

Create the VirtualBox VM

Install VirtualBox (and the Extension Pack) from virtualbox.org if it isn't already on this PC, then create a new VM.

1. Download **Ubuntu Server 22.04 LTS** ISO from ubuntu.com/download/server.
2. In VirtualBox: **Machine** → **New**. Name it `algo-pbx`, type **Linux**, version **Ubuntu (64-bit)**.
3. Memory: **4096 MB minimum** (8192 recommended — Postgres + Asterisk + Coturn + Next.js together). CPU: 2+ cores.
4. Disk: create a new VDI, **40 GB minimum**, dynamically allocated.
5. **Settings** → **Network** → **Adapter 1** → **Attached to: Bridged Adapter** (not NAT). This gives the VM a real IP on your LAN — required so the Dinstar gateway and any SIP phones on the same network can reach it directly, and so it can later get a public IP/port-forward for the domain.
6. **Settings** → **Storage** → attach the Ubuntu Server ISO to the optical drive.

REFERENCE · VIRTUALBOX "CREATE VIRTUAL MACHINE" WIZARD

WHY BRIDGED, NOT NAT

Later steps route SIP/RTP traffic and Tailscale subnet routes to/from this VM. NAT mode hides the VM behind VirtualBox's own virtual router and breaks inbound reachability. Bridged puts the VM directly on your physical LAN.

STEP 2

Install Ubuntu Server 22.04

Boot the VM. The Subiquity installer walks through these screens in order — accept defaults except where noted.

Screen	What to select
Language	English (or your preference)
Keyboard	Match your physical layout
Network connections	Confirm the bridged adapter picked up a DHCP address from your router — note this IP, you'll SSH to it
Proxy / Mirror	Leave blank / default
Storage	**Use an entire disk** → the VDI you created
Profile setup	Set a real username/password — you'll need it for `sudo`
SSH Setup	**Check "Install OpenSSH server"** — this is the one screen people miss, and without it you can't SSH in afterward
Featured snaps	Skip all — install Docker manually in Step 3, not as a snap

REFERENCE · UBUNTU INSTALLER — SSH SETUP SCREEN

Reboot when prompted, remove the ISO if VirtualBox doesn't do it automatically (**Devices → Optical Drives → Remove disk**), and log in once at the console to confirm it boots.

> **FROM THIS POINT ON, WORK OVER SSH**
> From your Windows PC: `ssh youruser@<vm-lan-ip>`. This lets you copy/paste the exact commands below instead of retyping them in the VirtualBox console.

STEP 3

Prepare the VM — Docker and firewall

Per this repo's `DEPLOYMENT.md` §1, the VM needs Docker Engine + Compose v2 and a specific set of open ports.

Install Docker Engine + Compose v2

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y ca-certificates curl gnupg

sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/key
sudo chmod a+r /etc/apt/keyrings/docker.gpg

echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo "$VERSION_CODENAME")
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compos

# Run docker without sudo:
sudo usermod -aG docker $USER
newgrp docker

docker --version
docker compose version
```

Open the required ports

Straight from `DEPLOYMENT.md` §1 — these are the ports Algo PBX needs reachable:

Port(s)	Protocol	Purpose
80, 443	TCP	Web app (behind your own reverse proxy — this repo does not ship one)
8089	TCP (WSS)	WebRTC signaling to browser softphones
5060	UDP	SIP to the Dinstar trunk
10000-20000	UDP	Asterisk RTP media
20001-30000	UDP	Coturn TURN relay

```
sudo ufw allow 22/tcp
sudo ufw allow 80,443/tcp
sudo ufw allow 8089/tcp
sudo ufw allow 5060/udp
sudo ufw allow 10000:20000/udp
sudo ufw allow 20001:30000/udp
sudo ufw enable
sudo ufw status
```

IF THIS VM SITS BEHIND A ROUTER (HOME/OFFICE LAN)

Bridged mode gives the VM a LAN IP, but reaching it from the public internet still needs the router's own port-forwarding rules for the same port list, forwarded to the VM's LAN IP. Do this once you know the VM's final LAN IP (Step 2).

STEP 4

Clone the repo and generate secrets

This section is `DEPLOYMENT.md` §2, verbatim.

```
git clone <your-fork-url> algo-pbx
cd algo-pbx
cp .env.example .env
```

Generate the bootstrap secrets:

```
openssl rand -hex 32      # → SETTINGS_ENCRYPTION_KEY
openssl rand -base64 33   # → AUTH_SECRET
openssl rand -hex 32      # → COTURN_AUTH_SECRET
openssl rand -hex 32      # → AMI_SECRET      (generate once per line)
openssl rand -hex 32      # → CDR_AMI_SECRET
openssl rand -hex 32      # → CDR_INGEST_SECRET
```

Open `.env` and fill in the **bootstrap section** (above the `RUNTIME SETTINGS` marker):

Variable	Value
`POSTGRES_USER` / `POSTGRES_PASSWORD` / `POSTGRES_DB`	Pick your own — not the placeholders
`SETTINGS_ENCRYPTION_KEY`	From `openssl rand -hex 32` above
`AUTH_SECRET` / `AUTH_URL`	Secret above; URL is your final `https://<domain>`
`AMI_USERNAME` / `AMI_SECRET`, `CDR_AMI_USERNAME` / `CDR_AMI_SECRET`	Secrets above
`VM_PUBLIC_DOMAIN` / `VM_PUBLIC_IP` / `VM_PRIVATE_IP`	See below — needs the domain from the companion PDF
`COTURN_AUTH_SECRET`	Secret above
`DINSTAR_LAN_IP`	See companion PDF §2 (Dinstar setup)

```
# On the VM itself:
curl ifconfig.me                    # → VM_PUBLIC_IP
ip route get 1 | awk '{print $7}'   # → VM_PRIVATE_IP
```

> **DO NOT FILL IN RUNTIME SETTINGS YET**
>
> Resend, OpenWA, Meta, Dinstar SMS credentials, Firebase, and the CRM webhook secret are all configurable later from `/admin/settings`, with no restart. Leave them blank unless you specifically want a fully non-interactive first deploy — see the companion PDF.

STEP 5

Generate the media/TLS certificates

From `pbx_configs/keys/README.md` — the stack will build without these, but no call connects until they exist.

DTLS-SRTP media cert (self-signed is fine)

```
ast_tls_cert -C ${VM_PUBLIC_DOMAIN} -O "Algo IT" -d ./pbx_configs/keys

# or, if ast_tls_cert isn't on the host:
openssl req -x509 -newkey rsa:2048 -nodes -days 3650 \
  -keyout pbx_configs/keys/asterisk.key \
  -out pbx_configs/keys/asterisk.crt \
  -subj "/CN=${VM_PUBLIC_DOMAIN}"
```

`dtls_verify=fingerprint` on every PJSIP endpoint validates the SDP fingerprint in signaling, not the certificate's chain of trust — self-signed is correct here, don't spend a real cert on it.

Real WSS signaling cert — must be CA-issued

Unlike the DTLS pair, `fullchain.pem` / `privkey.pem` (used by `http.conf` for the WSS transport on 8089) must be a real, browser-trusted certificate — browsers refuse a `wss://` connection to an untrusted cert outright, with no "accept the risk" click-through. The companion PDF's domain section covers getting this cert once the domain is pointed at this VM (either via Let's Encrypt, or Cloudflare's Origin CA certificate if you're proxying through Cloudflare).

```
# Once you have a cert (Let's Encrypt example):
sudo cp /etc/letsencrypt/live/<domain>/fullchain.pem pbx_configs/keys/
sudo cp /etc/letsencrypt/live/<domain>/privkey.pem   pbx_configs/keys/
```

None of these five files are committed to git (`.gitignore` already excludes them) — generate them on the VM itself.

STEP 6

Build and start the stack

```
docker compose build
docker compose up -d
docker compose logs -f web   # watch for a clean startup, Ctrl+C once stable
```

The `web` container runs `prisma migrate deploy` automatically on start — the schema is created on first boot, no manual migration step.

WHAT A CLEAN STARTUP LOOKS LIKE IN THE LOGS

```
web_1 | Prisma schema loaded, 0 migrations pending... applying
web_1 | Migrations applied successfully
web_1 | ▲ Next.js ready in 842ms
web_1 | - Local: http://localhost:3000
asterisk_1 | Asterisk Ready.
```

Quick sanity checks:

```
docker compose ps                # all services "Up"
docker compose logs asterisk | tail -30
docker compose logs coturn | tail -20
```

STEP 7

First login and hand-off

Visit `https://<your-domain>/setup` in a browser (once DNS/TLS from the companion PDF are in place — until then, `http://<vm-lan-ip>:3000/setup` works for a same-network smoke test).

- This page is reachable only until the first `ADMIN` account exists — visiting it afterward redirects to `/login`.
- It refuses to proceed if `SETTINGS_ENCRYPTION_KEY` wasn't set in Step 4, and names the exact fix command if so.
- Creating the admin account drops you straight into `/admin/settings` — that's where the companion PDF picks up.

ONGOING OPERATIONS, FOR LATER

Updates: `git pull && docker compose build && docker compose up -d` — migrations run automatically. Credential rotation happens in `/admin/settings`, no redeploy. Telephony config (AMI, Coturn, `VM_PUBLIC_DOMAIN`) lives only in `.env/pbx_configs/` and needs a restart, by design.

BEFORE YOUR FIRST `GIT PUSH`

Run `git status` and confirm nothing under `.env`, `pbx_configs/keys/`, `recordings/`, `voicemail/`, or `agent-photos/` is staged. `.gitignore` already excludes all of these — this is a sanity check, not a fix step.

Continue in **"Configuring Credentials, Dinstar & Going Live"** for `/admin/settings`, the Dinstar UC2000 gateway, and pointing your domain at this VM.